Data： 2018年度A1交通事故資料

Data Source： 政府資料開放平台

Link： https://data.gov.tw/dataset/158862

統資三甲 陳彥嘉 412421446

評分項目：1

Data Preprocessing ：

Data Acquisition & Feature Dropping:

```python
# 2.
import pandas as pd

a = r"C:\Users\Charles\OneDrive\桌面\2018年度A1交通事故資料.csv"
df = pd.read_csv(a, encoding='utf-8-sig')


drop_list = [
    '發生年度','發生月份','發生日期','發生時間','事故類別名稱',
    '處理單位名稱警局層','發生地點','道路型態子類別名稱','事故位置子類別名稱',
    '路面狀況-路面鋪裝名稱','路面狀況-路面缺陷名稱','道路障礙-障礙物名稱', # 刪除影響較小的障礙物
    '道路障礙-視距品質名稱','道路障礙-視距名稱','號誌-號誌種類名稱',
    '號誌-號誌動作名稱','車道劃分設施-分向設施大類別名稱','車道劃分設施-分向設施子類別名稱',
    '車道劃分設施-分道設施-快車道或一般車道間名稱','車道劃分設施-分道設施-快慢車道間名稱',
    '車道劃分設施-分道設施-路面邊線名稱','事故類型及型態大類別名稱','事故類型及型態子類別名稱',
    '肇因研判大類別名稱-主要','肇因研判子類別名稱-主要','死亡受傷人數','當事者順位',
    '當事者區分-類別-子類別名稱-車種', '保護裝備名稱', '行動電話或電腦或其他相類功能裝置名稱',
    '當事者行動狀態大類別名稱', '當事者行動狀態子類別名稱', '車輛撞擊部位名稱',
    '車輛撞擊部位子類別名稱','肇因研判大類別名稱-個別','肇因研判子類別名稱-個別','經度','緯度',
    '車輛撞擊部位子類別名稱-其他','車輛撞擊部位子類別名稱-最初','車輛撞擊部位大類別名稱-其他','車輛撞擊部位大類別名稱-最初'
    ]

df.drop(columns=drop_list,inplace=True, errors='ignore')
target_y='當事者區分-類別-大類別名稱-車種'
#df = df.dropna(subset=[target_y]).copy()
df = df.dropna(subset=[target_y]).copy()
df.info()
```

Feature Dropping

Variables :

```
Index: 3256 entries, 0 to 3674
Data columns (total 11 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   天候名稱                  3256 non-null   object
 1   光線名稱                  3256 non-null   object
 2   道路類別-第1當事者-名稱        3256 non-null   object
 3   速限-第1當事者              3256 non-null   float64
 4   道路型態大類別名稱            3256 non-null   object
 5   事故位置大類別名稱            3256 non-null   object
 6   路面狀況-路面狀態名稱          3256 non-null   object
 7   當事者區分-類別-大類別名稱-車種  3256 non-null   object
 8   當事者屬-性-別名稱           3256 non-null   object
 9   當事者事故發生時年齡          3256 non-null   float64
 10  肇事逃逸類別名稱-是否肇逃       3256 non-null   object
```

Target Variable

Data Imputation :

```python
df['當事者事故發生時年齡'] = pd.to_numeric(df['當事者事故發生時年齡'], errors='coerce')
df['當事者事故發生時年齡'] = df['當事者事故發生時年齡'].fillna(df['當事者事故發生時年齡'].mean())
df['速限-第1當事者'] = df['速限-第1當事者'].fillna(df['速限-第1當事者'].mean())
```

評分項目：2、3 、4

Decision Tree :

```python
print("目標變數分布：")
print(pd.Series(y).value_counts())

ohe = OneHotEncoder(sparse_output=False)


Weather = ohe.fit_transform(df[["天候名稱"]].fillna("未知"))
Weather = pd.DataFrame(Weather, index=df.index)
Weather.columns = ["Weather_" + str(s) for s in ohe.categories_[0]]


Light = ohe.fit_transform(df[["光線名稱"]].fillna("未知"))
Light = pd.DataFrame(Light, index=df.index)
Light.columns = ["Light_" + str(s) for s in ohe.categories_[0]]


RoadType = ohe.fit_transform(df[["道路類別-第1當事者-名稱"]].fillna("未知"))
RoadType = pd.DataFrame(RoadType, index=df.index)
RoadType.columns = ["RoadType_" + str(s) for s in ohe.categories_[0]]


RoadShape = ohe.fit_transform(df[["道路型態大類別名稱"]].fillna("未知"))
RoadShape = pd.DataFrame(RoadShape, index=df.index)
RoadShape.columns = ["RoadShape_" + str(s) for s in ohe.categories_[0]]


Surface = ohe.fit_transform(df[["路面狀況-路面狀態名稱"]].fillna("未知"))
Surface = pd.DataFrame(Surface, index=df.index)
Surface.columns = ["Surface_" + str(s) for s in ohe.categories_[0]]


Sex = ohe.fit_transform(df[["當事者屬-性-別名稱"]].fillna("未知"))
Sex = pd.DataFrame(Sex, index=df.index)
Sex.columns = ["Sex_" + str(s) for s in ohe.categories_[0]]

run = ohe.fit_transform(df[["肇事逃逸類別名稱-是否肇逃"]].fillna("未知"))
run= pd.DataFrame(run, index=df.index)
run.columns = ["run_" + str(s) for s in ohe.categories_[0]]

place = ohe.fit_transform(df[["事故位置大類別名稱"]].fillna("未知"))
place = pd.DataFrame(place, index=df.index)
place.columns = ["place_" + str(s) for s in ohe.categories_[0]]
```

```python
X_train, X_test, y_train, y_test = train_test_split(X_new, y,
                                                    test_size=0.2, random_state=20251224)
```

```python
df["y_binary"] = df[target_y].apply(
    lambda x: 1 if "機車" in str(x) else 0)
y = df["y_binary"].values
```

評分項目：5、6、7

Decision Tree :

```
#8 決策樹部分-請使用全變數實測過度學 習 最小資料筆數=2,最小分割請從0.1~0.01 確認從哪裡開始過度學習
acc=[]
for i in range(10,0,-1):

    clf= DecisionTreeClassifier(criterion="gini",min_samples_leaf=2,
                                min_samples_split=i/100,random_state=20251224)
    #配適建模資料集
    clf.fit(X_train,y_train)
    #用clf.score 列出正確率
    print("train Acc=",clf.score(X_train,y_train))
    print("test Acc=",clf.score(X_test,y_test))
    print("Leaves of the tree=", clf.get_n_leaves())
    print("Depth of the tree=", clf.get_depth())
    acc.append([i/100,clf.score(X_train,y_train),clf.score(X_test,y_test),clf.get_n_leaves(),clf.get_depth()])
```

Index	split	training	testing	leaves	depth
0	0.1	0.695084	0.642638	22	11
1	0.09	0.695084	0.642638	23	11
2	0.08	0.697005	0.641104	27	12
3	0.07	0.697389	0.625767	35	15
4	0.06	0.700461	0.619632	39	15
5	0.05	0.711214	0.621166	51	17
6	0.04	0.72043	0.634969	62	17
7	0.03	0.72427	0.633436	79	19
8	0.02	0.736943	0.624233	114	19
9	0.01	0.769585	0.616564	212	22

評分項目 : 8

Decision Tree :

```python
print("\n--- [方法 A: Chi-square 挑選後結果] ---")
clf_chi_ent = DecisionTreeClassifier(criterion="entropy", min_samples_split=0.08,
                                     min_samples_leaf=2, random_state=20251224)
clf_chi_ent.fit(X_train_chi, y_train)
print(f"Entropy (Chi2) 訓練 Acc: {clf_chi_ent.score(X_train_chi, y_train):.4f}")
print(f"Entropy (Chi2) 測試 Acc: {clf_chi_ent.score(X_test_chi, y_test):.4f}")

clf_chi_ent2 = DecisionTreeClassifier(criterion="gini", min_samples_split=0.08,
                                      min_samples_leaf=2, random_state=20251224)
clf_chi_ent2.fit(X_train_chi, y_train)
print(f"gini (Chi2) 訓練 Acc: {clf_chi_ent.score(X_train_chi, y_train):.4f}")
print(f"gini (Chi2) 測試 Acc: {clf_chi_ent.score(X_test_chi, y_test):.4f}")
```

```
--- [方法 A: Chi-square 挑選後結果] ---
Entropy (Chi2) 訓練 Acc: 0.6847
Entropy (Chi2) 測試 Acc: 0.6518
gini (Chi2) 訓練 Acc: 0.6847
gini (Chi2) 測試 Acc: 0.6518
```

評分項目 : 9、10

Decision Tree :

```python
print("\n--- [評分項目 11: 全變數 vs 挑選變數比較結果] ---")

# 建立兩棵樹進行對照，統一參數：深度=8
clf_all_vars = DecisionTreeClassifier(criterion="entropy", max_depth=8,
                                      min_samples_split=0.08, random_state=20251224)
clf_sel_vars = DecisionTreeClassifier(criterion="entropy", max_depth=8,
                                      min_samples_split=0.08, random_state=20251224)

# 全變數訓練
clf_all_vars.fit(X_train, y_train)
# 挑選後的 5 個變數訓練
clf_sel_vars.fit(X_train[top5_cols], y_train)

print(f"使用的重要變數: {top5_cols}")
print(f"全變數 (Depth=8) 測試正確率: {clf_all_vars.score(X_test, y_test):.4f}")
print(f"挑選後 (Depth=8) 測試正確率: {clf_sel_vars.score(X_test[top5_cols], y_test):.4f}")
```

```
--- [評分項目 11: 全變數 vs 挑選變數比較結果] ---
使用的重要變數: ['當事者事故發生時年齡', 'RoadType_國道', 'Light_夜間(或隧道、
地下道、涵洞)有照明', '速限-第1當事者', 'RoadType_省道']
全變數 (Depth=8) 測試正確率: 0.6273
挑選後 (Depth=8) 測試正確率: 0.6380
```

```
--- [方法 A: Chi-square 挑選後結果] ---
Entropy (Chi2) 訓練 Acc: 0.6847
Entropy (Chi2) 測試 Acc: 0.6518
gini (Chi2) 訓練 Acc: 0.6847
gini (Chi2) 測試 Acc: 0.6518
```

評分項目 :11

Decision Tree :

```python
print("\n--- [評分項目 12: 建模建議] ---")
# 我們建議使用「變數挑選後」且「深度=8」的模型
suggested_model = clf_sel_vars

print(f"建議模型：變數挑選決策樹 (Top 5 Features)")
print(f"樹的深度 (Depth): {suggested_model.get_depth()}")
print(f"葉子總數 (Number of Leaves): {suggested_model.get_n_leaves()}")
print(f"總節點數 (Tree Size): {suggested_model.tree_.node_count}")

# 正確率回顧
print(f"測試正確率: {suggested_model.score(X_test[top5_cols], y_test):.4f}")
```

```
--- [評分項目 12: 建模建議] ---
建議模型：變數挑選決策樹 (Top 5 Features)
樹的深度 (Depth): 8
葉子總數 (Number of Leaves): 22
總節點數 (Tree Size): 43
測試正確率: 0.6380
```

評分項目：12

Decision Tree :

Age > 60:
If the accident occurs at an intersection and at night with street lighting, the model is more likely to classify it as a motorcycle-related case.

Age < 15:
Below the legal age for holding a motorcycle license, the case is classified as non-motorcycle (likely a passenger or pedestrian).

Age > 88:
Accidents involving extremely elderly individuals are generally classified as non-motorcycle-related.

評分項目 :13

```
|--- RoadType_國道 <= 0.50
|  |--- 當事者事故發生時年齡 <= 29.50
|  |  |--- 當事者事故發生時年齡 <= 14.50
|  |  |  |--- class: 0
|  |  |--- 當事者事故發生時年齡 >  14.50
|  |  |  |--- 當事者事故發生時年齡 <= 24.50
|  |  |  |  |--- RoadType_省道 <= 0.50
|  |  |  |  |  |--- 速限-第1當事者 <= 65.00
|  |  |  |  |  |  |--- 速限-第1當事者 <= 35.00
|  |  |  |  |  |  |  |--- class: 1
|  |  |  |  |  |  |--- 速限-第1當事者 >  35.00
|  |  |  |  |  |  |  |--- 當事者事故發生時年齡 <= 19.50
|  |  |  |  |  |  |  |  |--- class: 1
|  |  |  |  |  |  |  |--- 當事者事故發生時年齡 >  19.50
|  |  |  |  |  |  |  |  |--- class: 1
|  |  |  |  |  |--- 速限-第1當事者 >  65.00
|  |  |  |  |  |  |--- class: 0
|  |  |  |  |--- RoadType_省道 >  0.50
|  |  |  |  |  |--- class: 0
|  |  |  |--- 當事者事故發生時年齡 >  24.50
|  |  |  |  |--- class: 0
|  |--- 當事者事故發生時年齡 >  29.50
|  |  |--- 當事者事故發生時年齡 <= 58.50
|  |  |  |--- RoadType_省道 <= 0.50
|  |  |  |  |--- 速限-第1當事者 <= 45.00
|  |  |  |  |  |--- class: 0
|  |  |  |  |--- 速限-第1當事者 >  45.00
|  |  |  |  |  |--- 當事者事故發生時年齡 <= 37.50
|  |  |  |  |  |  |--- 速限-第1當事者 <= 65.00
|  |  |  |  |  |  |  |--- class: 0
|  |  |  |  |  |  |--- 速限-第1當事者 >  65.00
|  |  |  |  |  |  |  |--- class: 1
|  |  |  |  |  |--- 當事者事故發生時年齡 >  37.50
|  |  |  |  |  |  |--- 速限-第1當事者 <= 75.00
|  |  |  |  |  |  |  |--- 當事者事故發生時年齡 <= 49.50
|  |  |  |  |  |  |  |  |--- class: 0
|  |  |  |  |  |  |  |--- 當事者事故發生時年齡 >  49.50
|  |  |  |  |  |  |  |  |--- class: 0
|  |  |  |  |  |  |--- 速限-第1當事者 >  75.00
|  |  |  |  |  |  |  |--- class: 0
|  |  |  |--- RoadType_省道 >  0.50
|  |  |  |  |--- class: 0
|  |  |--- 當事者事故發生時年齡 >  58.50
|  |  |  |--- Light_夜間(或隧道、地下道、涵洞)有照明 <= 0.50
|  |  |  |  |--- 當事者事故發生時年齡 <= 87.50
|  |  |  |  |  |--- 當事者事故發生時年齡 <= 69.50
|  |  |  |  |  |  |--- 速限-第1當事者 <= 45.00
|  |  |  |  |  |  |  |--- class: 0
|  |  |  |  |  |  |--- 速限-第1當事者 >  45.00
|  |  |  |  |  |  |  |--- class: 0
|  |  |  |  |  |--- 當事者事故發生時年齡 >  69.50
|  |  |  |  |  |  |--- 速限-第1當事者 <= 35.00
|  |  |  |  |  |  |  |--- class: 0
|  |  |  |  |  |  |--- 速限-第1當事者 >  35.00
|  |  |  |  |  |  |  |--- 當事者事故發生時年齡 <= 77.50
|  |  |  |  |  |  |  |  |--- class: 1
|  |  |  |  |  |  |  |--- 當事者事故發生時年齡 >  77.50
|  |  |  |  |  |  |  |  |--- class: 0
|  |  |  |  |--- 當事者事故發生時年齡 >  87.50
|  |  |  |  |  |--- class: 0
|  |  |  |--- Light_夜間(或隧道、地下道、涵洞)有照明 >  0.50
|  |  |  |  |--- class: 0
|--- RoadType_國道 >  0.50
|  |--- class: 0
```

Decision Tree :

```
# ======================================================================
# 補充評分項目 14：最佳模型之交叉驗證 CV=5
# ======================================================================
# 針對全變數模型做 CV=5
cv_scores = cross_val_score(clf_all_vars, X_new, y, cv=5)
print(f"\n--- [評分項目 14: 交叉驗證] ---")
print(f"全資料全變數 CV=5 平均正確率: {cv_scores.mean():.4f}") #14
```

```
--- [評分項目 14: 交叉驗證] ---
全資料全變數 CV=5 平均正確率: 0.6616
```

評分項目：14

Association Rules:

```
# 1. 執行 SMOTE
sm = SMOTE(random_state=20251224)
X_res, y_res = sm.fit_resample(X_new, y)
```

```
target
0    2073
1    2073
Name: count, dtype: int64
==============================
```

SMOTE

```
#把df_hot欄位中 pep欄位所有的值都改成小寫lower()
moto_cols=[c for c in df_hot.columns if "moto" in c.lower()]
#^ 把符合的條件丟到一個list
print("moto相關欄位=",moto_cols)
targets_y=None

for c in moto_cols:
    if "1" in c.lower():
        target_y = c
        break

if target_y is not None:
    print("目前我們要找的目標欄位moto=",1)
    rules_moto=rules[rules["consequents"].apply(lambda x:target_y in [str(i) for i in x])]
```

Association Rules

評分項目：15 、16

Association Rules:

rules - DataFrame

Index	antecedents	consequents	ntecedent suppo	nsequent suppo	support	confidence	lift	representativity	leverage	convicti
24988	frozenset({'當事者區分-類別-大類別名稱-車種_機車', '光線名稱_日間自然光線', '事故位置大類別名稱_交叉路口'})	frozenset({'肇事逃逸類別名稱-是否肇逃_否', '道路型態大類別名稱_交岔路', 'y_binary_1'})	0.12285	0.177826	0.12285	1	5.62349	1	0.101004	inf
24993	frozenset({'事故位置大類別名稱_交叉路口', '光線名稱_日間自然光線', 'y_binary_1'})	frozenset({'當事者區分-類別-大類別名稱-車種_機車', '道路型態大類別名稱_交岔路', '肇事逃逸類別名稱-是否肇逃_否'})	0.12285	0.177826	0.12285	1	5.62349	1	0.101004	inf
26910	frozenset({'當事者區分-類別-大類別名稱-車種_機車', '事故位置大類別名稱_交叉路口', '速限-第1當事者_50.0'})	frozenset({'肇事逃逸類別名稱-是否肇逃_否', '道路型態大類別名稱_交岔路', 'y_binary_1'})	0.112408	0.177826	0.112408	1	5.62349	1	0.0924189	inf
26915	frozenset({'事故位置大類別名稱_交叉路口', '速限-第1當事者_50.0', 'y_binary_1'})	frozenset({'當事者區分-類別-大類別名稱-車種_機車', '道路型態大類別名稱_交岔路', '肇事逃逸類別名稱-是否肇逃_否'})	0.112408	0.177826	0.112408	1	5.62349	1	0.0924189	inf
30707	frozenset({'當事者區分-類別-大類別名稱-車種_機車', '光線名稱_日間自然光線', '事故位置大類別名稱_交叉路口', '天候名稱_晴'})	frozenset({'肇事逃逸類別名稱-是否肇逃_否', '道路型態大類別名稱_交岔路', 'y_binary_1'})	0.107801	0.177826	0.107801	1	5.62349	1	0.0886312	inf
30712	frozenset({'事故位置大類別名稱_交叉路口', '光線名稱_日間自然光線', 'y_binary_1', '天候名稱_晴'})	frozenset({'當事者區分-類別-大類別名稱-車種_機車', '道路型態大類別名稱_交岔路', '肇事逃逸類別名稱-是否肇逃_否'})	0.107801	0.177826	0.107801	1	5.62349	1	0.0886312	inf
35476	frozenset({'當事者區分-類別-大類別名稱-車種_機車', '光線名稱_日…	frozenset({'肇事逃逸類別名稱-是否肇逃_否', '道路型態大類別名稱_交岔路', 'y_binary_1'})	0.117015	0.177826	0.117015	1	5.62349	1	0.0962065	inf
35481	frozenset({'事故位置大類別名稱_交叉路口', '光線名稱_日間自然光線', 'y_binary_1', '路面狀況-路面狀態名稱_乾燥'})	frozenset({'當事者區分-類別-大類別名稱-車種_機車', '道路型態大類別名稱_交岔路', '肇事逃逸類別名稱-是否肇逃_否'})	0.117015	0.177826	0.117015	1	5.62349	1	0.0962065	inf
35951	frozenset({'當事者區分-類別-大類別名稱-車種_機車', '路面狀況-路…	frozenset({'肇事逃逸類別名稱-是否肇逃_否', '道路型態大類別名稱_交岔路', 'y_binary_1'})	0.101966	0.177826	0.101966	1	5.62349	1	0.0838335	inf
35956	frozenset({'事故位置大類別名稱_交叉路口', '路面狀況-路面狀態名稱_乾燥', '速限-第1當事者_50.0', 'y_binary_1'})	frozenset({'當事者區分-類別-大類別名稱-車種_機車', '道路型態大類別名稱_交岔路', '肇事逃逸類別名稱-是否肇逃_否'})	0.101966	0.177826	0.101966	1	5.62349	1	0.0838335	inf
37059	frozenset({'光線名稱_日間自然光線', '當事者區分-類別-大類別名稱-…	frozenset({'肇事逃逸類別名稱-是否肇逃_否', '道路型態大類別名稱_交岔路', 'y_binary_1'})	0.107494	0.177826	0.107494	1	5.62349	1	0.0883787	inf
37064	frozenset({'光線名稱_日間自然光線', 'y_binary_1', '路面狀況-路面狀態名稱_乾燥', '事故位置大類別名稱_交叉路口', '天候名稱_晴'})	frozenset({'當事者區分-類別-大類別名稱-車種_機車', '道路型態大類別名稱_交岔路', '肇事逃逸類別名稱-是否肇逃_否'})	0.107494	0.177826	0.107494	1	5.62349	1	0.0883787	inf
4173	frozenset({'事故位置大類別名稱_交叉路口', 'y_binary_1'})	frozenset({'當事者區分-類別-大類別名稱-車種_機車', '道路型態大類別名稱_交岔路'})	0.173833	0.178747	0.173833	1	5.5945	1	0.142761	inf
4175	frozenset({'當事者區分-類別-大類別名稱-車種_機車', '事故位置大類別名稱_交叉路口'})	frozenset({'道路型態大類別名稱_交岔路', 'y_binary_1'})	0.173833	0.178747	0.173833	1	5.5945	1	0.142761	inf
8630	frozenset({'當事者區分-類別-大類別名稱-車種_機車', '事故位置大類別名稱_交叉路口', '天候名稱_晴'})	frozenset({'道路型態大類別名稱_交岔路', 'y_binary_1'})	0.144963	0.178747	0.144963	1	5.5945	1	0.119051	inf
8631	frozenset({'事故位置大類別名稱_交叉路口', 'y_binary_1', '天候名稱_晴'})	frozenset({'當事者區分-類別-大類別名稱-車種_機車', '道路型態大類別名稱_交岔路'})	0.144963	0.178747	0.144963	1	5.5945	1	0.119051	inf
10996	frozenset({'當事者區分-類別-大類別名稱-車種_機車', '光線名稱_日間自然光線', '事故位置大類別名稱_交叉路口'})	frozenset({'道路型態大類別名稱_交岔路', 'y_binary_1'})	0.12285	0.178747	0.12285	1	5.5945	1	0.100891	inf
10998	frozenset({'事故位置大類別名稱_交叉路口', '光線名稱_日間自然光線', 'y_binary_1'})	frozenset({'當事者區分-類別-大類別名稱-車種_機車', '道路型態大類別名稱_交岔路'})	0.12285	0.178747	0.12285	1	5.5945	1	0.100891	inf
12482	frozenset({'當事者區分-類別-大類別名稱-車種_機車', '道路類別-第1當事者-名稱_市區道路', '事故位置大類別名稱_交叉路口'})	frozenset({'道路型態大類別名稱_交岔路', 'y_binary_1'})	0.109951	0.178747	0.109951	1	5.5945	1	0.0902975	inf
12484	frozenset({'事故位置大類別名稱_交叉路口', '道路類別-第1當事者-名稱_市區道路', 'y_binary_1'})	frozenset({'當事者區分-類別-大類別名稱-車種_機車', '道路型態大類別名稱_交岔路'})	0.109951	0.178747	0.109951	1	5.5945	1	0.0902975	inf
13339	frozenset({'當事者區分-類別-大類別名稱-車種_機車', '事故位置大類別名稱_交叉路口', '速限-第1當事者_50.0'})	frozenset({'道路型態大類別名稱_交岔路', 'y_binary_1'})	0.112408	0.178747	0.112408	1	5.5945	1	0.0923153	inf
13341	frozenset({'事故位置大類別名稱_交叉路口', '速限-第1當事者_50.0', 'y_binary_1'})	frozenset({'當事者區分-類別-大類別名稱-車種_機車', '道路型態大類別名稱_交岔路'})	0.112408	0.178747	0.112408	1	5.5945	1	0.0923153	inf
13904	frozenset({'當事者區分-類別-大類別名稱-車種_機車', '路面狀況-路面	frozenset({'道路型態大類別名稱_交岔路', 'y_binary_1'})	0.160319	0.178747	0.160319	1	5.5945	1	0.131663	inf

Intersections have a relatively strong association with vehicle type, with both support and lift being high.

評分項目：15 、17

SVM :

```python
print("目標變數分布：")
print(pd.Series(y).value_counts())

ohe = OneHotEncoder(sparse_output=False)


Weather = ohe.fit_transform(df[["天候名稱"]].fillna("未知"))
Weather = pd.DataFrame(Weather, index=df.index)
Weather.columns = ["Weather_" + str(s) for s in ohe.categories_[0]]


Light = ohe.fit_transform(df[["光線名稱"]].fillna("未知"))
Light = pd.DataFrame(Light, index=df.index)
Light.columns = ["Light_" + str(s) for s in ohe.categories_[0]]


RoadType = ohe.fit_transform(df[["道路類別-第1當事者-名稱"]].fillna("未知"))
RoadType = pd.DataFrame(RoadType, index=df.index)
RoadType.columns = ["RoadType_" + str(s) for s in ohe.categories_[0]]


RoadShape = ohe.fit_transform(df[["道路型態大類別名稱"]].fillna("未知"))
RoadShape = pd.DataFrame(RoadShape, index=df.index)
RoadShape.columns = ["RoadShape_" + str(s) for s in ohe.categories_[0]]


Surface = ohe.fit_transform(df[["路面狀況-路面狀態名稱"]].fillna("未知"))
Surface = pd.DataFrame(Surface, index=df.index)
Surface.columns = ["Surface_" + str(s) for s in ohe.categories_[0]]


Sex = ohe.fit_transform(df[["當事者屬-性-別名稱"]].fillna("未知"))
Sex = pd.DataFrame(Sex, index=df.index)
Sex.columns = ["Sex_" + str(s) for s in ohe.categories_[0]]

run = ohe.fit_transform(df[["肇事逃逸類別名稱-是否肇逃"]].fillna("未知"))
run= pd.DataFrame(run, index=df.index)
run.columns = ["run_" + str(s) for s in ohe.categories_[0]]

place = ohe.fit_transform(df[["事故位置大類別名稱"]].fillna("未知"))
place = pd.DataFrame(place, index=df.index)
place.columns = ["place_" + str(s) for s in ohe.categories_[0]]
```

```python
X_train, X_test, y_train, y_test = train_test_split(X_new, y,
                                                    test_size=0.2, random_state=20251224)
```

```python
df["y_binary"] = df[target_y].apply(
    lambda x: 1 if "機車" in str(x) else 0)
y = df["y_binary"].values
```

```python
ss = StandardScaler()
X_train_std = ss.fit_transform(X_train)
X_test_std = ss.transform(X_test)
```

評分項目：18 、 19、20、21

SVM：

```
訓練資料集正確率= 0.5602918586789555
測試資料集正確率= 0.558282208588957
訓練分錯幾個= 1145
測試分錯幾個= 288
訓練F1_score= 0.5647880458932715
訓練F1_score= 0.5573344858110324
訓練資料集正確率= 0.5602918586789555
測試資料集正確率= 0.558282208588957
訓練分錯幾個= 1145
測試分錯幾個= 288
訓練F1_score= 0.5647880458932715
訓練F1_score= 0.5573344858110324
核函數 SVM_train Acc= 0.7073732718894009
核函數 SVM_test Acc= 0.6273006134969326
核函數 SVM_train Acc= 0.7315668202764977
核函數 SVM_test Acc= 0.6211656441717791
```

```python
#svm 18 - 23 用線性 acc比較高
X_train_std=ss.transform(X_train)
X_test_std=ss.transform(X_test)

from sklearn.svm import LinearSVC
m=LinearSVC(C=0.1,class_weight="balanced")
m.fit(X_train_std,y_train)

print("訓練資料集正確率=",m.score(X_train_std, y_train))
print("測試資料集正確率=",m.score(X_test_std, y_test))

y_pred_train=m.predict(X_train_std)
print("訓練分錯幾個=",(y_train!=y_pred_train).sum())
y_pred_test=m.predict(X_test_std)
print("測試分錯幾個=",(y_test!=y_pred_test).sum())

from sklearn.metrics import f1_score
print("訓練F1_score=",f1_score(y_train,y_pred_train,average="weighted"))
print("訓練F1_score=",f1_score(y_test,y_pred_test,average="weighted"))

m1=LinearSVC(C=0.2,class_weight="balanced")
m1.fit(X_train_std,y_train)

print("訓練資料集正確率=",m1.score(X_train_std, y_train))
print("測試資料集正確率=",m1.score(X_test_std, y_test))

y_pred_train=m1.predict(X_train_std)
print("訓練分錯幾個=",(y_train!=y_pred_train).sum())
y_pred_test=m1.predict(X_test_std)
print("測試分錯幾個=",(y_test!=y_pred_test).sum())

from sklearn.metrics import f1_score
print("訓練F1_score=",f1_score(y_train,y_pred_train,average="weighted"))
print("訓練F1_score=",f1_score(y_test,y_pred_test,average="weighted"))

from sklearn.svm import SVC
svm=SVC(gamma=0.1,C=1.0,kernel="rbf",probability=True,random_state=20251217)
svm2=SVC(gamma=0.1,C=2.0,kernel="rbf",probability=True,random_state=20251217)

svm.fit(X_train_std,y_train)
print("核函數 SVM_train Acc=",svm.score(X_train_std,y_train))
print("核函數 SVM_test Acc=",svm.score(X_test_std,y_test))
svm2.fit(X_train_std,y_train)
print("核函數 SVM_train Acc=",svm2.score(X_train_std,y_train))
print("核函數 SVM_test Acc=",svm2.score(X_test_std,y_test))
```

評分項目 22 、23

Random Forest :

```python
print("目標變數分布：")
print(pd.Series(y).value_counts())

ohe = OneHotEncoder(sparse_output=False)


Weather = ohe.fit_transform(df[["天候名稱"]].fillna("未知"))
Weather = pd.DataFrame(Weather, index=df.index)
Weather.columns = ["Weather_" + str(s) for s in ohe.categories_[0]]


Light = ohe.fit_transform(df[["光線名稱"]].fillna("未知"))
Light = pd.DataFrame(Light, index=df.index)
Light.columns = ["Light_" + str(s) for s in ohe.categories_[0]]


RoadType = ohe.fit_transform(df[["道路類別-第1當事者-名稱"]].fillna("未知"))
RoadType = pd.DataFrame(RoadType, index=df.index)
RoadType.columns = ["RoadType_" + str(s) for s in ohe.categories_[0]]


RoadShape = ohe.fit_transform(df[["道路型態大類別名稱"]].fillna("未知"))
RoadShape = pd.DataFrame(RoadShape, index=df.index)
RoadShape.columns = ["RoadShape_" + str(s) for s in ohe.categories_[0]]


Surface = ohe.fit_transform(df[["路面狀況-路面狀態名稱"]].fillna("未知"))
Surface = pd.DataFrame(Surface, index=df.index)
Surface.columns = ["Surface_" + str(s) for s in ohe.categories_[0]]


Sex = ohe.fit_transform(df[["當事者屬-性-別名稱"]].fillna("未知"))
Sex = pd.DataFrame(Sex, index=df.index)
Sex.columns = ["Sex_" + str(s) for s in ohe.categories_[0]]

run = ohe.fit_transform(df[["肇事逃逸類別名稱-是否肇逃"]].fillna("未知"))
run= pd.DataFrame(run, index=df.index)
run.columns = ["run_" + str(s) for s in ohe.categories_[0]]

place = ohe.fit_transform(df[["事故位置大類別名稱"]].fillna("未知"))
place = pd.DataFrame(place, index=df.index)
place.columns = ["place_" + str(s) for s in ohe.categories_[0]]
```

```python
X_train, X_test, y_train, y_test = train_test_split(X_new, y,
                                                    test_size=0.2, random_state=20251224)
```

```python
df["y_binary"] = df[target_y].apply(
    lambda x: 1 if "機車" in str(x) else 0)
y = df["y_binary"].values
```

```python
ss = StandardScaler()
X_train_std = ss.fit_transform(X_train)
X_test_std = ss.transform(X_test)
```

```python
#因為測試資料及代表母體 也就是未知資料 在標準化的時候不應加入 只有訓練資料標準化
X_train[["當事者事故發生時年齡"]]=ss.fit_transform(X_train[["當事者事故發生時年齡"]])
X_test[["當事者事故發生時年齡"]]=ss.transform(X_test[["當事者事故發生時年齡"]])


X_train[["速限-第1當事者"]]=ss.fit_transform(X_train[["速限-第1當事者"]])
X_test[["速限-第1當事者"]]=ss.transform(X_test[["速限-第1當事者"]])
```

評分項目：24、25 、26 、27

Random Forest :

```python
#rf
from sklearn.ensemble import RandomForestClassifier
RF=RandomForestClassifier(n_estimators=100, max_depth=8,
                          random_state=20251224)

RF.fit(X_train,y_train)
print("隨機森林 100 train acc=",RF.score(X_train,y_train))
print("隨機森林 100 test acc=",RF.score(X_test,y_test))

from sklearn.ensemble import RandomForestClassifier
RF2=RandomForestClassifier(n_estimators=200, max_depth=8,
                          random_state=20251224)

RF2.fit(X_train,y_train)
print("隨機森林 200 train acc=",RF2.score(X_train,y_train))
print("隨機森林 200 test acc=",RF2.score(X_test,y_test))

from sklearn.ensemble import RandomForestClassifier
RF3=RandomForestClassifier(n_estimators=300, max_depth=8,
                          random_state=20251224)

RF3.fit(X_train,y_train)
print("隨機森林 300 train acc=",RF3.score(X_train,y_train))
print("隨機森林 300 test acc=",RF3.score(X_test,y_test))

from sklearn.ensemble import RandomForestClassifier
RF4=RandomForestClassifier(n_estimators=500, max_depth=8,
                          random_state=20251224)

RF4.fit(X_train,y_train)
print("隨機森林 500 train acc=",RF4.score(X_train,y_train))
print("隨機森林 500 test acc=",RF4.score(X_test,y_test))
```

```
隨機森林 100 train acc= 0.7142857142857143
隨機森林 100 test acc= 0.6165644171779141
隨機森林 200 train acc= 0.7142857142857143
隨機森林 200 test acc= 0.6211656441717791
隨機森林 300 train acc= 0.7150537634408602
隨機森林 300 test acc= 0.6226993865030674
隨機森林 500 train acc= 0.7162058371735791
隨機森林 500 test acc= 0.6180981595092024
```

Since the accuracy difference is small,
the simpler model is chosen to prevent overfitting.

評分項目：28 、29

KNN：

```
print("目標變數分布：")
print(pd.Series(y).value_counts())

ohe = OneHotEncoder(sparse_output=False)


Weather = ohe.fit_transform(df[["天候名稱"]].fillna("未知"))
Weather = pd.DataFrame(Weather, index=df.index)
Weather.columns = ["Weather_" + str(s) for s in ohe.categories_[0]]


Light = ohe.fit_transform(df[["光線名稱"]].fillna("未知"))
Light = pd.DataFrame(Light, index=df.index)
Light.columns = ["Light_" + str(s) for s in ohe.categories_[0]]


RoadType = ohe.fit_transform(df[["道路類別-第1當事者-名稱"]].fillna("未知"))
RoadType = pd.DataFrame(RoadType, index=df.index)
RoadType.columns = ["RoadType_" + str(s) for s in ohe.categories_[0]]


RoadShape = ohe.fit_transform(df[["道路型態大類別名稱"]].fillna("未知"))
RoadShape = pd.DataFrame(RoadShape, index=df.index)
RoadShape.columns = ["RoadShape_" + str(s) for s in ohe.categories_[0]]


Surface = ohe.fit_transform(df[["路面狀況-路面狀態名稱"]].fillna("未知"))
Surface = pd.DataFrame(Surface, index=df.index)
Surface.columns = ["Surface_" + str(s) for s in ohe.categories_[0]]


Sex = ohe.fit_transform(df[["當事者屬-性-別名稱"]].fillna("未知"))
Sex = pd.DataFrame(Sex, index=df.index)
Sex.columns = ["Sex_" + str(s) for s in ohe.categories_[0]]

run = ohe.fit_transform(df[["肇事逃逸類別名稱-是否肇逃"]].fillna("未知"))
run= pd.DataFrame(run, index=df.index)
run.columns = ["run_" + str(s) for s in ohe.categories_[0]]

place = ohe.fit_transform(df[["事故位置大類別名稱"]].fillna("未知"))
place = pd.DataFrame(place, index=df.index)
place.columns = ["place_" + str(s) for s in ohe.categories_[0]]
```

```
X_train, X_test, y_train, y_test = train_test_split(X_new, y,
                                                    test_size=0.2, random_state=20251224)
```

```
df["y_binary"] = df[target_y].apply(
    lambda x: 1 if "機車" in str(x) else 0)
y = df["y_binary"].values
```

```
ss = StandardScaler()
X_train_std = ss.fit_transform(X_train)
X_test_std = ss.transform(X_test)
```

評分項目：30 、31 、32 、33

KNN：

```
#利用迴圈來找最好的K
acc=[]
#用i控制最近鄰
for i in range(1,2604): #2941
    knn=KNeighborsClassifier(n_neighbors=i)
    knn.fit(X_train,y_train)
    print("K=",i,"測試正確率=",knn.score(X_test, y_test))
    acc.append(knn.score(X_test, y_test))

print("測試正確率最高的是=",max(acc))
#因為acc是從0開始編號 k=1會在acc[0]
#所以迴圈從0~480 但是bestK要加個1
bestK=0

for i in range (0,2603):
        if acc[i]==max(acc):
            bestK=i+1

print("測試正確率最高的是=",max(acc),"k=",bestK)
```

```
K= 2591 測試正確率= 0.6119631901840491
K= 2592 測試正確率= 0.6119631901840491
K= 2593 測試正確率= 0.6119631901840491
K= 2594 測試正確率= 0.6119631901840491
K= 2595 測試正確率= 0.6119631901840491
K= 2596 測試正確率= 0.6119631901840491
K= 2597 測試正確率= 0.6119631901840491
K= 2598 測試正確率= 0.6119631901840491
K= 2599 測試正確率= 0.6119631901840491
K= 2600 測試正確率= 0.6119631901840491
K= 2601 測試正確率= 0.6119631901840491
K= 2602 測試正確率= 0.6119631901840491
K= 2603 測試正確率= 0.6119631901840491
測試正確率最高的是= 0.6687116564417178
測試正確率最高的是= 0.6687116564417178 k= 32
```

評分項目：34、35

Voting :

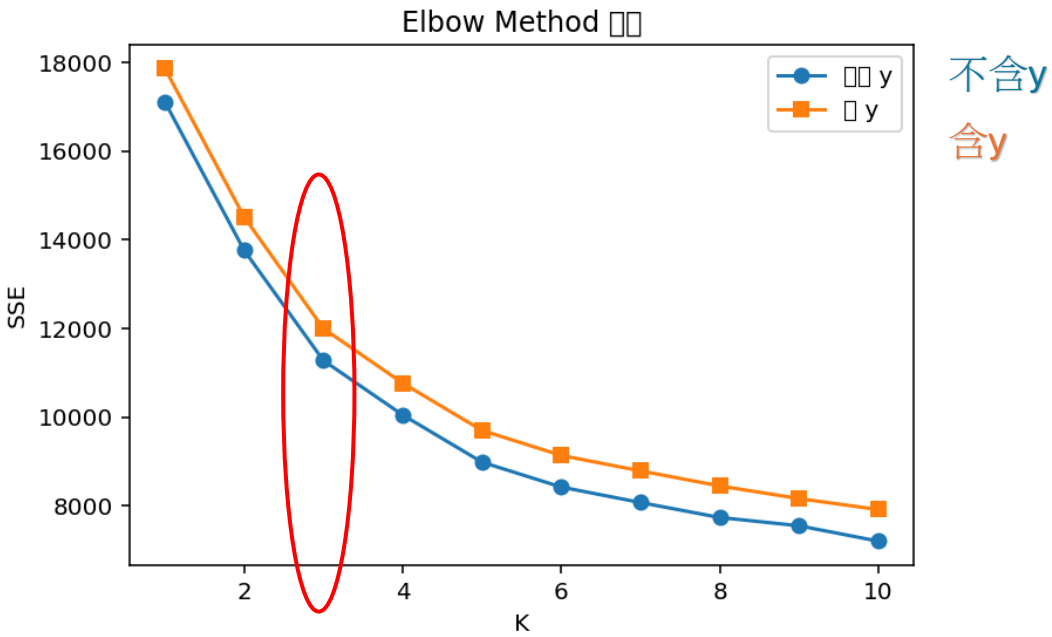
```python
print("決策樹 train Acc=",clf.score(X_train_std,y_train))
print("決策樹 test Acc=",clf.score(X_test_std,y_test))
print("RF200 train Acc=",RF.score(X_train_std,y_train))
print("RF200 test Acc=",RF.score(X_test_std,y_test))
print("KNN_train acc=",knn.score(X_train_std,y_train))
print("KNN_test acc=",knn.score(X_test_std,y_test))
print("核函數 SVM_train Acc=",svm.score(X_train_std,y_train))
print("核函數 SVM_test Acc=",svm.score(X_test_std,y_test))


from sklearn.ensemble import VotingClassifier
voting_hard=VotingClassifier(estimators=[
   ("支援向量機",svm),("決策數",clf),("K最近鄰",knn),("隨機森林",RF)],
    voting="hard",n_jobs=-1)

voting_hard.fit(X_train_std,y_train)

print("voting hard_train acc=",voting_hard.score(X_train_std,y_train))
print("voting hard_test acc=",voting_hard.score(X_test_std,y_test))


#做voting 按照測試資料集的高低 排序模型順序
from sklearn.ensemble import VotingClassifier
voting_soft=VotingClassifier(estimators=[
    ("支援向量機",svm),("決策數",clf),("K最近鄰",knn),("隨機森林",RF)],
    voting="soft",n_jobs=-1)

voting_soft.fit(X_train_std,y_train)

print("voting Soft_train acc=",voting_soft.score(X_train_std,y_train))
print("voting Soft_test acc=",voting_soft.score(X_test_std,y_test))
```

```
決策樹 train Acc= 0.6954685099846 39
決策樹 test Acc= 0.6257668711656442
RF200 train Acc= 0.6466973886328725
RF200 test Acc= 0.6088957055214724
KNN_train acc= 0.7519201228878648
KNN_test acc= 0.6150306748466258
核函數 SVM_train Acc= 0.7073732718894009
核函數 SVM_test Acc= 0.6273006134969326
voting hard_train acc= 0.717357910906298
voting hard_test acc= 0.6180981595092024
voting Soft_train acc= 0.7403993855606759
voting Soft_test acc= 0.6288343558282209
```

評分項目 : 36、37

K-means :

```python
# ====== 複製 X_new 來做 K-means (不含 y) ======
X_no_y = X_new.copy()
X_no_y['當事者事故發生時年齡'] = scale(X_no_y['當事者事故發生時年齡'])
X_no_y['速限-第1當事者'] = scale(X_no_y['速限-第1當事者'])

# ====== 複製 X_new 來做 K-means (含 y) ======
X_with_y = X_no_y.copy()
X_with_y['y_binary'] = df['y_binary'].values

# ========================
# 計算 SSE (Elbow) 比較
# ========================
SSE_no_y = []
SSE_with_y = []

for k in range(1, 11):
    km = KMeans(n_clusters=k, init="k-means++", n_init=10, random_state=20251224)
    km.fit(X_no_y)
    SSE_no_y.append(km.inertia_)

    km.fit(X_with_y)
    SSE_with_y.append(km.inertia_)

plt.plot(range(1, 11), SSE_no_y, marker="o", label="不含 y")
plt.plot(range(1, 11), SSE_with_y, marker="s", label="含 y")
plt.xlabel("K")
plt.ylabel("SSE")
plt.title("Elbow Method 比較")
plt.legend()
plt.show()

print("SSE 不含 y =", SSE_no_y)
print("SSE 含 y =", SSE_with_y)
```



評分項目：39 、40、41

K-means:

```python
# ========================
# 假設最佳群數 K=4
# ========================
K_best = 4

# 不含 y
km_no_y = KMeans(n_clusters=K_best, n_init=10, random_state=20251224)
labels_no_y = km_no_y.fit_predict(X_no_y)
sil_no_y = silhouette_score(X_no_y, labels_no_y)
print("\n不含 y 輪廓係數 =", format(sil_no_y, ".4f"))

# 含 y
km_with_y = KMeans(n_clusters=K_best, n_init=10, random_state=20251224)
labels_with_y = km_with_y.fit_predict(X_with_y)
sil_with_y = silhouette_score(X_with_y, labels_with_y)
print("含 y 輪廓係數 =", format(sil_with_y, ".4f"))
```

```
不含 y 輪廓係數 = 0.2014
含 y 輪廓係數 = 0.1908
```

評分項目：42

K-means:

```python
def pseudo_accuracy(labels, df_binary):
    df_tmp = pd.DataFrame()
    df_tmp['群組'] = labels
    df_tmp['車種二元'] = df_binary.map({0: "非機車", 1: "機車"})
    ctab = pd.crosstab(df_tmp['車種二元'], df_tmp['群組'])
    print("\nK-means 分群 × 機車/非機車 交叉矩陣：")
    print(ctab)
    correct = sum(ctab.max())
    total = ctab.values.sum()
    acc = correct / total
    return acc

acc_no_y = pseudo_accuracy(labels_no_y, df['y_binary'])
print("不含 y 用分群來預測分類的正確率 =",(512+718+133+487)/2882)

acc_with_y = pseudo_accuracy(labels_with_y, df['y_binary'])
print("含 y 用分群來預測分類的正確率 =", (492+718+507+133)/2882)
```

```
K-means 分群 × 機車/非機車 交叉矩陣：
群組      0    1    2    3
車種二元
機車    258  409   78  287
非機車   512  718  133  487
不含 y 用分群來預測分類的正確率 = 0.6419153365718251

K-means 分群 × 機車/非機車 交叉矩陣：
群組      0    1    2    3
車種二元
機車    244  408  302   78
非機車   492  718  507  133
含 y 用分群來預測分類的正確率 = 0.6419153365718251
```

評分項目：43 、45

K-means:

Index	者事故發生時	即限-第1當事者	Weather_強風	Weather_晴	Weather_暴雨	Weather_陰	Weather_雨	Weather_霧或煙	道、地下道	道、地下道	ht_日間自然光	ight_晨或暮光	RoadType_其他	RoadType_國道	adType_專用道	dType_市區道	dType_村里
群組 0：高風險路口群	24.453	50.0129	0.000862813	0.786022	0.00258844	0.117343	0.0931838	9.75782e-19	0.406385	0.0215703	0.534081	0.0379638	0.0224331	0.00172563	0.00517688	0.571182	0.151855
群組 1：快車道	71.6703	50.4105	-6.50521e-19	0.802696	0.00245098	0.106618	0.0882353	-1.30104e-18	0.226716	0.0196078	0.697304	0.0563725	0.0318627	0.00122549	0.00367647	0.506127	0.202206
群組 2：高齡者生活圈群	40.5847	100.645	5.42101e-20	0.705645	-8.67362e-19	0.0685484	0.225806	1.0842e-19	0.241935	0.330645	0.423387	0.00403226	0.00403226	0.842742	-1.73472e-18	0.0282258	0.0322581
群組 3：夜間/特定環境群	48.1704	49.6854	5.42101e-20	0.78122	0.000968054	0.11423	0.101646	0.00193611	0.296225	0.0280736	0.633107	0.0425944	0.0319458	0.00677638	0.00387222	0.555663	0.162633

dType_村里道	RoadType_省道	RoadType_縣道	RoadType_鄉道	RoadShape_交岔路	oadShape_單路部分	oadShape_圓環廣場	RoadShape_平交道	Surface_乾燥	Surface_冰雪	Surface_濕潤	Sex_女	無或物(動物、堆	Sex_男
0.151855	0.121657	0.08283	0.0431406	0.452977	0.545298	0.00172563	9.75782e-19	0.874892	0.00258844	0.122519	0.21484	0.000862813	0.784297
0.202206	0.131127	0.0723039	0.0514706	0.57598	0.419118	0.00490196	-1.30104e-18	0.886029	0.00122549	0.112745	0.366422	-6.50521e-19	0.633578
0.0322581	0.0927419	-8.32667e-17	-4.16334e-17	-5.55112e-16	1	4.33681e-19	1.0842e-19	0.754032	-8.67362e-19	0.245968	0.157258	5.42101e-20	0.842742
0.162633	0.127783	0.0696999	0.0416263	0.495644	0.500484	0.00193611	0.00193611	0.869313	0.00193611	0.128751	0.228461	5.42101e-20	0.771539

run_否	run_是	place_交叉路口	place_交流道	place_其他	place_路段
0.988783	0.0112166	0.438309	0.00172563	0.0224331	0.537532
0.987745	0.0122549	0.552696	7.80626e-18	0.0281863	0.419118
0.987903	0.0120968	5.55112e-16	0.016129	-2.08167e-17	0.983871
0.981607	0.018393	0.476283	0.00677638	0.0271055	0.489835

```
kmeans=KMeans(n_clusters=4,init="k-means++",random_state=20251224)
kmeans.fit(X_new)
df1=pd.DataFrame(kmeans.cluster_centers_)
df1.columns=X_new.columns
```

Cluster 0: High-Risk Intersection Group (Motorcycles / Young to Middle-Aged Riders)
Higher values in RoadShape_Intersection, with an average age between 20–40.
The cross-tabulation shows a large number of non-motorcycles as well, indicating frequent car–motorcycle collisions at intersections.

Cluster 1: High-Speed Road Group (Mostly Cars / Non-Motorcycles)
Higher SpeedLimit values (around 50–70 km/h),Road types are more likely highways or suburban roads.
This explains why this cluster has the largest number of non-motorcycle cases (718).
On high-speed roads, accidents involving cars or larger vehicles are more common.

Cluster 2: Elderly Residential Area Group
The average age in this cluster is significantly higher than others (over 60 years old).
This is the smallest cluster, suggesting that accidents involving elderly people occur less frequently and are mostly non-motorcycle-related.

Cluster 3: Nighttime / Special Environment Group
Higher values in Light_Night with Lighting or specific weather conditions.
This cluster represents accidents occurring under limited visibility or special environments, and is also mainly non-motorcycle-related.

評分項目：43、44

Best model: K-means without y

```
K-means 分群 × 機車/非機車 交叉矩陣：
群組      0    1    2    3
車種二元
機車    258  409   78  287
非機車   512  718  133  487
不含 y 用分群來預測分類的正確率 = 0.6419153365718251
```

Has high acc

can look at the details within each cluster

評分項目 : 46